HACKATHON

Paradigmas de Programação na Prática

Sistema de Gestão de Cinema

Disciplina	Professor	Modalidade	Nível
Paradigmas de Programação	Prof. Me. José Antonio Gallo Junior	Hackathon em Grupos	Engenharia da Computação

1. Apresentação da Atividade

Esta atividade tem formato de Hackathon: um desafio colaborativo em que os grupos trabalharão de forma progressiva, construindo um sistema de gestão de um cinema. Cada fase acrescenta novos requisitos ao sistema anterior, revelando, na prática, as limitações do paradigma imperativo puro e a elegância da programação procedural.

Ao final, os grupos terão vivenciado na prática a transição entre paradigmas e estarão preparados para compreender, em aula futura, como a Orientação a Objetos resolve os desafios que surgirem aqui.

Objetivo Pedagógico

Ao longo das fases, os alunos irão perceber que:

- O paradigma imperativo funciona bem para problemas simples e lineares.
- Conforme a complexidade cresce, o código imperativo fica difuso, extenso e difícil de manter.
- A programação procedural (uso de funções/procedimentos) organiza a lógica, elimina repetição e torna o sistema mais legível e escalável.
- A Orientação a Objetos (próxima aula) vai além, modelando múltiplas salas e entidades de forma natural.

2. Organização da Atividade

2.1 Formação dos Grupos

- Grupos de 3 a 4 integrantes.
- Cada grupo deve eleger um porta-voz para apresentar a solução ao final de cada fase.
- Todos os integrantes devem participar ativamente da codificação.

2.2 Ferramentas

- Ambiente de desenvolvimento: Portugol Studio (https://portugol.dev)
- Linguagem: Portugol (português estruturado)
- Não é permitido o uso de materiais prontos da internet.
- Consultas ao professor são bem-vindas a qualquer momento.

2.3 Estrutura Geral

O hackathon está dividido em 4 fases progressivas. Cada fase tem uma duração sugerida. Ao final de cada fase, o professor fará uma breve pausa para que um grupo apresente sua solução e o turma discuta os pontos-chave.

Fase	Título	Conceito-Chave
1	Venda de Ingressos (Básico)	Variáveis, constantes, I/O
2	Desconto para Estudantes	Estrutura condicional (se/senão)
3	Atendimento de Múltiplos Clientes	Estrutura de repetição (enquanto)
4	Controle de Poltronas	Vetores, matrizes e procedural

FASE 1

Venda de Ingressos — O Básico

Paradigma Imperativo | Variáveis, Constantes e Entrada/Saída

Contexto

O Cinema GalloMax acabou de abrir as portas e precisa de um sistema para registrar a venda de ingressos. Por enquanto, o sistema precisa ser simples: recebe a quantidade de ingressos desejada e informa o valor total a pagar.

Requisitos

- O preço padrão de cada ingresso é R$ 22,50.
- O sistema deve perguntar ao cliente quantos ingressos deseja comprar.
- O sistema deve calcular e exibir o valor total da compra.

Exemplo de Execução Esperada

```
-- CINEMA GALLOMAX --

Sistema de Vendas de Ingressos
Informe a quantidade de ingressos a vender: 3

Valor Total: R$ 67.50
```

Dicas para o Grupo

- Declare uma constante para o preço (real PRECO = 22.50). Isso evita o uso de 'números mágicos' no código.
- Use variáveis para armazenar a quantidade e o total calculado.
- Operação de cálculo: total = quantidade * PRECO.

Reflexão ao Final da Fase 1

Converse com seu grupo:

- O código ficou simples e direto. Por quê?
- O que aconteceria se o preço mudasse? Onde você precisaria alterar o código?
- O uso da constante ajudou? Como?

FASE 2

Desconto para Estudantes

Paradigma Imperativo | Estrutura Condicional

Contexto

Ótimo! O sistema básico funciona. Mas o Cinema GalloMax anunciou uma promoção: estudantes têm direito a 50% de desconto no ingresso. O sistema agora precisa identificar se o cliente é estudante e aplicar o desconto correspondente.

Novos Requisitos (acrescentar ao Fase 1)

- Após informar a quantidade de ingressos, o sistema pergunta se os ingressos são para estudantes.
- Se sim, o valor total é dividido por 2 (50% de desconto).
- Exibir o valor final com ou sem desconto conforme a resposta.

Exemplo de Execução Esperada

```
-- CINEMA GALLOMAX --

Sistema de Vendas de Ingressos
Informe a quantidade de ingressos a vender: 3
Esses ingressos sao para estudantes? verdadeiro

Valor Total: R$ 33.75
```

Dicas para o Grupo

- Use uma variável lógica (logico estudante) para armazenar a resposta.
- Utilize a estrutura: se (estudante) { total = total / 2 }
- O restante do código da Fase 1 permanece igual.

Reflexão ao Final da Fase 2

Converse com seu grupo:

- Quantas linhas foram necessárias para adicionar a lógica do desconto?
- O código ainda está legível? Você consegue entendê-lo de um lado para o outro?
- E se houvesse mais tipos de desconto (idosos, crianças, convênios)? O que aconteceria com o código?

FASE 3

Atendimento de Múltiplos Clientes

Paradigma Imperativo | Estrutura de Repetição e Relatório

Contexto

O cinema abre todos os dias e precisa atender muitos clientes em sequência, não apenas um. O sistema agora deve continuar atendendo novos clientes até que o operador decida encerrar o atendimento. Ao final, deve exibir um relatório consolidado da sessão.

Novos Requisitos (acrescentar ao Fase 2)

- Após cada venda, o sistema pergunta: "Atender novo cliente?"
- Enquanto a resposta for verdadeira, o sistema repete o processo de venda.
- Ao encerrar, o sistema exibe um relatório final com:
  - Total de clientes atendidos
  - Total de ingressos vendidos
  - Quantidade de clientes estudantes
  - Valor total arrecadado na sessão

Exemplo de Execução Esperada

```
-- CINEMA GALLOMAX --

Sistema de Vendas de Ingressos
Atendendo ao 1 cliente
Informe a quantidade de ingressos a vender: 2
Esses ingressos sao para estudantes? falso
Valor Total: R$ 45.00

Atender novo cliente? verdadeiro

Atendendo ao 2 cliente
Informe a quantidade de ingressos a vender: 1
Esses ingressos sao para estudantes? verdadeiro
Valor Total: R$ 11.25

Atender novo cliente? falso

-- RELATORIO FINAL --
Total de Clientes Atendidos: 2
Total de Estudantes........: 1
Total de Ingressos Vendidos: 3
Lucro Total ...............: R$ 56.25
```

Dicas para o Grupo

- Use a estrutura enquanto (continuar) { ... } para o laço principal.
- Declare acumuladores antes do laço: cliente, totalIngressos, totalGeral, qtdeEstudantes.
- Incremente os acumuladores dentro do laço a cada atendimento.
- Exiba o relatório após o laço terminar.

Reflexão ao Final da Fase 3 — Ponto Central do Hackathon

Esta é a reflexão mais importante da atividade. Converse com seu grupo:

- O programa agora tem quantas linhas? Continua fácil de entender?
- Se alguém que não participou do desenvolvimento precisasse alterar o cálculo do desconto, onde ele precisaria olhar? Seria fácil encontrar?
- Se o gerente pedisse um novo tipo de relatório, o que você precisaria modificar? Você conseguiria fazer isso sem quebrar o resto do código?
- Imagine que a lógica de venda precisasse ser repetida em outros pontos do sistema. O que você faria? Copiar o código? Isso é sustentável?

>> Esse é o momento em que o paradigma imperativo começa a mostrar seus limites. <<<

FASE 4

Controle de Poltronas — Vetores, Matrizes e Procedural

Paradigma Procedural | Funções, Vetores e Matrizes

Contexto

O cinema decidiu modernizar ainda mais o sistema. Agora é necessário controlar as poltronas da sala individualmente. A sala possui 10 fileiras (A a J) e 20 poltronas por fileira. O cliente escolhe qual poltrona deseja, e o sistema verifica se ela está disponível. Ao final, o sistema exibe um mapa visual da sala.

Requisitos

- A sala é representada por uma matriz lógica de 10 x 20 (fileiras x poltronas).
- Ao iniciar, todas as poltronas estão livres.
- Durante a venda, para cada ingresso o cliente informa a fileira (A-J) e o número da poltrona (1-20).
- Se a poltrona estiver ocupada, o sistema avisa e pede outra escolha.
- Se estiver livre, a poltrona é marcada como ocupada e o ingresso é registrado.
- O sistema deve exibir o mapa da sala atualizado a cada atendimento.
- O relatório final continua com as mesmas informações da Fase 3.

Mapa Visual da Sala (exemplo)

```
   [  1][  2][  3][  4][  5][  6][  7][  8][  9][ 10][ 11][ 12][ 13][ 14][ 15][ 16][ 17][ 18][ 19][ 20]
A  [   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ]
B  [   ][ X ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ]
C  [   ][   ][   ][ X ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ]
D  [   ][   ][   ][ X ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ]
E  [   ][   ][   ][ X ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ]
F  [   ][   ][   ][ X ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ]
G  [   ][   ][   ][ X ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ]
H  [   ][   ][   ][ X ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ]
I  [   ][   ][   ][ X ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ]
J  [   ][   ][   ][ X ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ][   ]
```

O Grande Desafio: Organize com Procedural!

Perceba a complexidade que surgiu. Agora vocês precisam lidar com: desenhar a sala, marcar poltronas, encontrar o índice de uma fileira, registrar a venda e gerar o relatório. Se tudo isso ficar no bloco inicio(), o código se tornará impossível de ler e manter.

A solução é organizar o código em funções especializadas, conforme o paradigma procedural. Cada função tem uma responsabilidade clara e pode ser chamada de qualquer ponto do programa:

Função	Responsabilidade
`esvaziarSala()`	Inicializa toda a matriz com falso (todas as poltronas livres)
`desenharSala()`	Exibe no console o mapa visual da sala com [X] para ocupadas e [ ] para livres
`indiceFileira(fil)`	Recebe um caractere (ex: 'B') e retorna o índice numérico correspondente na matriz
`marcarPoltrona(fil, pol)`	Recebe fileira e poltrona, verifica disponibilidade e marca como ocupada; retorna verdadeiro/falso
`venderIngressos(qtde)`	Coordena o processo de venda: lê as poltronas, chama marcarPoltrona e contabiliza os valores
`relatorioFinal()`	Exibe o relatório consolidado da sessão ao encerrar o atendimento

Dicas para o Grupo

- Declare a matriz no escopo global: logico sala[FILEIRAS][POLTRONA]
- Declare também as constantes: inteiro FILEIRAS = 10 e inteiro POLTRONA = 20
- Declare os acumuladores (totalIngressos, qtdeClientes etc.) no escopo global para que todas as funções possam acessá-los.
- Comece implementando esvaziarSala() e desenharSala() — são as mais simples e permitem validar visualmente.
- Só depois implemente marcarPoltrona() e venderIngressos().

Reflexão ao Final da Fase 4

- Compare o código desta fase com o da Fase 3. Quantas linhas tem o início() agora?
- Se precisasse alterar a forma de desenhar a sala, onde você mexeria? Em um único lugar!
- Se um bug surgir na marcação de poltronas, você sabe exatamente onde olhar: marcarPoltrona().
- O código ficou mais longo no total, mas cada parte ficou mais simples e compreensível.
- Isso é a essência do paradigma procedural: dividir para conquistar.

3. Quadro Comparativo: Imperativo vs. Procedural

Ao finalizar as quatro fases, use este quadro para consolidar as diferenças percebidas na prática:

Paradigma Imperativo — Limitações	Paradigma Procedural — Vantagens
Toda a lógica em um único bloco	Lógica dividida em funções especializadas
Difícil de localizar e corrigir erros	Erros isolados na função responsável
Repetição de código quando há lógica similar	Reutilização: chame a função quantas vezes precisar
Qualquer mudança pode afetar todo o programa	Mudanças localizadas: altere apenas a função afetada
Difícil de testar partes isoladas	Cada função pode ser testada de forma independente
Código cresce de forma desordenada	Estrutura organizada e escalável

4. Próximos Passos: Rumo à Orientação a Objetos

Ao término deste hackathon, vocês vivenciaram as limitações do imperativo e a organização do procedural. Mas ainda resta um desafio que o procedural não resolve com elegância: e se o cinema tivesse múltiplas salas funcionando ao mesmo tempo?

Com o paradigma procedural, teríamos que criar variáveis globais separadas para cada sala (sala1[][], sala2[][], totalSala1, totalSala2...), o que rapidamente se tornaria inviável.

Teaser: Orientação a Objetos

Na próxima aula, veremos como o paradigma Orientado a Objetos resolve esse problema com elegância:

- Cada sala vira um objeto da classe Sala, com seus próprios atributos (poltronas, ingressos vendidos, receita).
- O cinema pode instanciar quantas salas quiser: sala1 = nova Sala(10, 20), sala2 = nova Sala(8, 15)...
- Os métodos da classe encapsulam a lógica: sala1.venderIngresso(), sala2.exibirMapa().
- A complexidade do sistema cresce, mas o código permanece organizado, legível e escalável.

Pergunta para reflexão antes da próxima aula:

Como você modelaria um cinema com 5 salas diferentes usando apenas o que aprendeu hoje?

Anote suas ideias e traga para discussão!